Create a Custom Implementation of the Promise API

Problem: Build `MyPromise`, a from-scratch implementation of the JavaScript Promise API (constructor, `then`, `catch`, `finally`, and the statics `resolve`, `reject`, `all`, `allSettled`, `race`, `any`).
Tech stack: Vanilla JavaScript (ES2022, runs in Node and the browser). The default React stack does not apply: this is a pure-logic problem with no UI.
Time limit: 60 minutes (default).

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements:** restate the problem, split functional from non-functional, ask clarifying questions, write down assumptions.
2. **List the interactions / use cases:** every input and the exact output it must give. This becomes your checklist and your test list.
3. **Design the architecture:** the classes / modules and one-line responsibility each.
4. **Design the state:** the smallest set of fields that describes the system at any moment.
5. **Design the contracts:** the public API (signatures, return values, errors).
6. **Trace the data flow** for one key interaction end to end.
7. **Find the hard parts** before you code, so you budget time for them.
8. **Build in layers:** simplest working version first, then add one working layer at a time, testing as you go.
9. **Cover states and edge cases.**
10. **Wrap up:** what you handled, the trade-offs, what you would add with more time.

Time budget (60 minutes)

Phase	Time
Requirements, clarifying questions, assumptions	5 min
Use cases, state, contracts, data flow	7 min
Core: constructor, resolve/reject, state machine	10 min
`then` with chaining + async callbacks	12 min
Resolution procedure (thenables, cycles)	6 min
`catch`, `finally`, `resolve`, `reject`	5 min
`all`, `allSettled`, `race`, `any`	10 min
Quick tests + wrap-up	5 min

Think out loud

Say every decision before you make it: "*I am storing callbacks in a queue because* `then` *can be called many times before the promise settles.*" The interviewer grades how you reason, not only whether the code runs. If you get stuck, say what you are trying and why: silence looks like being lost.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- `new MyPromise(executor)` runs `executor(resolve, reject)` **synchronously**.
- A promise has three states: `pending`, `fulfilled`, `rejected`. It settles **once**; later calls to `resolve` / `reject` are ignored.
- If the executor throws, the promise rejects with that error.
- `then(onFulfilled, onRejected)` returns a **new** promise, so calls can be chained.
- Callbacks run **asynchronously** (as microtasks), even when the promise is already settled.
- The value returned from a callback resolves the next promise; a throw rejects it.
- Returning a promise (or any thenable) from a callback makes the next promise **wait for it**.
- Missing callbacks pass the value / reason through the chain.
- `catch(fn)` = `then(undefined, fn)`.
- `finally(fn)` runs `fn` on either outcome, keeps the original value / reason, and waits if `fn` returns a promise.
- Statics: `resolve`, `reject`, `all`, `allSettled`, `race`, `any`.

Non-functional requirements

Concern	What it means here
Correctness	Follow the Promises/A+ spec; same callback ordering as native `Promise`.
Performance	O(1) per settle and per `then`; no polling, no `setTimeout` delays.
Scalability	Long chains and thousands of `then` calls on one promise must work (queue, not single slot).
Error handling	Executor throws, callback throws, thenable getter throws, chaining cycle, non-iterable input to statics.
Usability	Same API as native, so it is a drop-in replacement; works with `async/await`.
Interop	Must adopt native promises and any object with a `then` method.
Extensibility	New statics (e.g. `withResolvers`) are built on top of `then` without touching the core.

Clarifying questions

1. Must it be **Promises/A+ compliant**, or is a basic `then` chain enough?
2. Should callbacks run as **microtasks** (like native), or is `setTimeout` acceptable?
3. Which **statics** are in scope: only `resolve` / `reject` / `all`, or also `allSettled`, `race`, `any`?
4. Should it **interoperate** with native promises and third-party thenables?
5. Do we need **unhandled rejection** tracking (warnings for rejections nobody catches)?
6. Should it support **subclassing** (`class MyAjax extends MyPromise`) with `Symbol.species`?

Assumptions we will build with

- Full Promises/A+ compliance, verified with the official `promises-aplus-tests` suite.
- Callbacks scheduled with `queueMicrotask` (same queue native promises use).
- All six statics plus `catch` and `finally`.
- Interop with native promises and thenables: yes.
- No unhandled rejection tracking, no `Symbol.species` subclassing (mention as trade-offs).

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
`new MyPromise((res) => res(1))`	Fulfilled with `1`.
`new MyPromise(() => { throw err })`	Rejected with `err`.
`new MyPromise()` (no executor)	Throws `TypeError` synchronously.
`res(1); rej(2); res(3)` inside executor	Fulfilled with `1`; later calls ignored.
`p.then(cb)` on a settled promise	`cb` runs in a microtask, **after** current sync code.
`p.then(a); p.then(b)` before settle	Both run, in registration order.
`p.then(x => x + 1)`	New promise fulfilled with `x + 1`.
`p.then(() => { throw e })`	New promise rejected with `e`.
`p.then(() => otherPromise)`	New promise waits and adopts `otherPromise`'s result.
`p.then()` / `p.then(null)`	Value passes through unchanged.
`rejected.then(onOk)` (no `onRejected`)	Rejection passes through to the next `catch`.
`const q = p.then(() => q)`	`q` rejects with `TypeError` (chaining cycle).
`resolve({ then(r) { r(3) } })`	Fulfilled with `3` (thenable adopted).
`resolve(nativePromise)`	Adopts the native promise's result.
`p.finally(fn)`	`fn()` runs with no args; result keeps `p`'s value / reason.
`p.finally(() => { throw e })`	Result rejects with `e` (overrides).

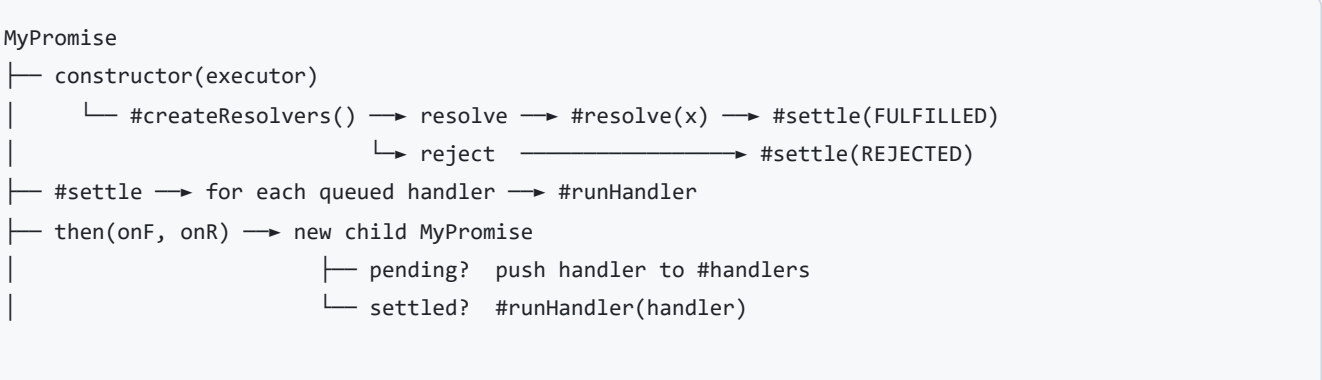
Action / input	Expected behavior / output
`MyPromise.all([p1, 'v', p3])`	Array of results in **input order**; rejects on first rejection.
`MyPromise.all([])`	Fulfilled with `[]` immediately.
`MyPromise.all(42)`	Rejected with `TypeError` (not iterable).
`MyPromise.allSettled([...])`	Never rejects; array of `{status, value}` / `{status, reason}`.
`MyPromise.race([...])`	Settles like the first input to settle. `race([])` stays pending forever.
`MyPromise.any([...])`	Fulfills with the first fulfilled; if all reject, `AggregateError` with all reasons.
`MyPromise.any([])`	Rejected with `AggregateError`.
`await myPromise`	Works, because `await` calls `.then`.

---

3. Design the architecture

One class is enough: the Promise API is a single object with a small state machine. We split responsibilities into **private methods**, not files.

Piece	Responsibility
`constructor`	Validate executor, create the one-shot `resolve` / `reject` pair, run executor in `try/catch`.
`#createResolvers()`	Return a `resolve` / `reject` pair that shares one `called` flag, so only the first call counts.
`#resolve(x)`	Promise Resolution Procedure: detect cycles, adopt thenables, otherwise fulfill.
`#settle(state, value)`	Move from `pending` to a final state once, then flush queued handlers.
`#runHandler(handler)`	Schedule a callback as a microtask and pipe its result into the child promise.
`then`	Create the child promise; queue the handler (pending) or run it (settled).
`catch`, `finally`	Thin wrappers over `then`.
Statics	Combinators built only on `MyPromise.resolve(...).then(...)`.



```
├── catch / finally ──► then
└── static resolve / reject / all / allSettled / race / any ──► resolve(...).then(...)
```

Start simple: first a version with no thenable adoption and no microtasks (sync callbacks), get chaining working, then add `queueMicrotask`, then the resolution procedure.

---

4. Design the state / data model

State or field	Purpose
`#state`	`'pending'`, `'fulfilled'` or `'rejected'`. Changes once.
`#value`	The fulfillment value **or** the rejection reason (one field, the state says which).
`#handlers`	Queue of `{ onFulfilled, onRejected, resolve, reject }` registered while pending.
`called` (closure)	One-shot guard shared by a `resolve` / `reject` pair.

```
const PENDING = 'pending';
const FULFILLED = 'fulfilled';
const REJECTED = 'rejected';

class MyPromise {
  #state = PENDING;
  #value = undefined;
  #handlers = []; // { onFulfilled, onRejected, resolve, reject }
}
```

Senior note

- **Derived, not stored:** "is settled" is `#state !== PENDING`; no separate boolean. One `#value` field serves both value and reason.
- **Who owns state:** only the promise itself can change its state. Outside code gets `resolve` / `reject` functions, never the fields: that is why they are `#private`.
- **Each handler owns its child:** a handler stores the **child promise's** `resolve` / `reject`, so the parent can drive the child without knowing anything else about it.
- `#handlers` **is cleared after settle:** once settled, new `then` calls run directly; keeping the old array would leak the callbacks.

---

5. Design the data structures and contracts

Example data

```
// A queued handler (one per then() call while pending)
{
```

```
  onFulfilled: (v) => v * 2,      // user callback or undefined
  onRejected: undefined,          // missing -> pass rejection through
  resolve: [Function],            // child promise's resolve
  reject: [Function],             // child promise's reject
}

// allSettled result items (same shape as native)
{ status: 'fulfilled', value: 1 }
{ status: 'rejected', reason: Error('no') }
```

No ids are needed: the handler queue is an **array**, so its order is the registration order, which the spec requires.

Contracts

API	Input	Output
`new MyPromise(executor)`	`executor(resolve, reject)`	Promise; `TypeError` if executor is not a function.
`then(onF?, onR?)`	Optional callbacks	**New** promise resolved with the callback's return value.
`catch(onR)`	Callback	`then(undefined, onR)`.
`finally(fn)`	Callback, called with no args	Promise with the **original** value / reason, unless `fn` throws or rejects.
`MyPromise.resolve(v)`	Any value	`v` itself if it is a `MyPromise`, else a promise adopting `v`.
`MyPromise.reject(r)`	Any reason	Rejected promise (never unwraps `r`).
`MyPromise.all(iter)`	Iterable	Values in input order, or the first rejection.
`MyPromise.allSettled(iter)`	Iterable	Array of status objects; never rejects.
`MyPromise.race(iter)`	Iterable	First settled outcome.
`MyPromise.any(iter)`	Iterable	First fulfillment, or `AggregateError(errors)`.

```
// Example usage (no real API: the "payload" is the value flowing through the chain)
MyPromise.resolve(2)
  .then((x) => x * 10)                  // 20
  .then((x) => new MyPromise((r) => setTimeout(() => r(x + 1), 100))) // waits -> 21
  .then((x) => { throw new Error(`bad ${x}`); })
  .catch((e) => e.message)              // 'bad 21'
  .finally(() => console.log('done'));  // value stays 'bad 21'
```

---

6. Trace the data flow

Key interaction: `p.then(cb)` where `p` is still pending and `cb` returns another promise.

```
p.then(cb)
   │
   ▼
then() creates child = new MyPromise((resolve, reject) => ...)
   │
   ▼
p is pending → push { onFulfilled: cb, resolve, reject } into p.#handlers
   │            (then() returns child immediately, sync code continues)
   ▼
later: executor calls resolve(10)
   │
   ▼
#createResolvers guard: first call? yes → called = true
   │
   ▼
#resolve(10): not a thenable → #settle(FULFILLED, 10)
   │
   ▼
#settle: state = fulfilled, value = 10 → #runHandler for each queued handler
   │
   ▼
#runHandler: queueMicrotask(...)   ← callback deferred until the call stack is empty
   │
   ▼
microtask: result = cb(10)  → returns promise q
   │
   ▼
child's resolve(q) → child.#resolve(q): q has a .then → queueMicrotask(q.then(resolve, reject))
   │
   ▼
q fulfills with 42 → child's inner resolve(42) → child.#settle(FULFILLED, 42)
   │
   ▼
child's own handlers run → the next .then in the chain receives 42
```

---

7. Important technical challenges

Challenge 1: Callbacks must always be asynchronous

Problem: `MyPromise.resolve(1).then(log); console.log('sync')`. If `then` runs `log` immediately because the promise is already settled, `1` prints **before** `sync`. Code would behave differently depending on whether the data was cached or not ("releasing Zalgo").

Solution: every callback goes through `queueMicrotask`, whether the promise is pending or settled. Microtasks run after the current sync code but **before** timers and rendering, which matches native ordering. `setTimeout` would work but is slower (macrotask, ~1-4 ms clamp) and gives a different order than native.

```
#runHandler({ onFulfilled, onRejected, resolve, reject }) {
  queueMicrotask(() => {
    const fulfilled = this.#state === FULFILLED;
    const callback = fulfilled ? onFulfilled : onRejected;
    if (typeof callback !== 'function') {
      return fulfilled ? resolve(this.#value) : reject(this.#value);
    }
    try {
      resolve(callback(this.#value));
    } catch (err) {
      reject(err);
    }
  });
}
```

Challenge 2: Chaining: `then` returns a new promise wired to the callback's result

Problem: `fetchUser().then(u => fetchPosts(u.id)).then(posts => ...)`. The second `then` must get the **posts**, not the promise returned by `fetchPosts`, and a throw anywhere must skip to the next `catch`. Returning `this` from `then` would break this: every `then` would see the same value.

Solution: `then` always creates a child promise and stores the **child's** `resolve` / `reject` with the callbacks. The callback's return value is fed to the child's `resolve` (which adopts promises, see challenge 3); a throw goes to the child's `reject`. A missing callback forwards the value or reason, which is how a rejection "skips" `then(onOk)` calls and lands in `catch`.

```
then(onFulfilled, onRejected) {
  return new MyPromise((resolve, reject) => {
    const handler = { onFulfilled, onRejected, resolve, reject };
    if (this.#state === PENDING) this.#handlers.push(handler);
    else this.#runHandler(handler);
  });
}
```

Challenge 3: The resolution procedure (thenables, cycles, one-shot calls)

Problem: `resolve(x)` cannot just store `x`:

- `x` may be a native promise or a library thenable (`{ then(res) { res(3) } }`): we must **wait** for it.
- Reading `x.then` may hit a **getter that throws**, or returns a different value each time.
- A malicious thenable may call `resolve` **and** `reject`, or call them many times.
- `const q = p.then(() => q)` would wait for itself forever.

Solution:

- Same object → reject with `TypeError`.
- Read `then` **once**, inside `try/catch`.
- If it is a function, call it with a **fresh one-shot** `resolve` / `reject` pair, so only the first call counts and a throw after a call is ignored.
- Call it in a microtask, the same as native's `NewPromiseResolveThenableJob`, so the ordering matches.

```
#resolve(x) {
  if (x === this) {
    return this.#settle(REJECTED, new TypeError('Chaining cycle detected for promise'));
  }
  if (x !== null && (typeof x === 'object' || typeof x === 'function')) {
    let then;
    try {
      then = x.then;
    } catch (err) {
      return this.#settle(REJECTED, err);
    }
    if (typeof then === 'function') {
      const { resolve, reject } = this.#createResolvers();
      queueMicrotask(() => {
        try {
          then.call(x, resolve, reject);
        } catch (err) {
          reject(err); // ignored if already called
        }
      });
      return;
    }
  }
  this.#settle(FULFILLED, x);
}
```

Challenge 4: Combinators: order, counting, and the empty case

Problem: `MyPromise.all([slow(20ms,'a'), 'b', fast(5ms,'c')])` must return `['a','b','c']`, not completion order `['b','c','a']`. Plain values (`'b'`) must work too. `all([])` must resolve, not hang. `all(42)` must reject, not throw synchronously.

Solution: capture the index `i` in the closure and write `results[i]`; count down `remaining` instead of checking `results.length` (an array with holes still has full length). Wrap each item with `MyPromise.resolve` so values and thenables are treated the same. Use `[...iterable]` inside the executor: it throws `TypeError` for non-iterables (`Array.from(42)` would silently return `[]`), and the executor's `try/catch` turns that into a rejection.

```
static all(iterable) {
  return new MyPromise((resolve, reject) => {
    const items = [...iterable];
    const results = new Array(items.length);
    let remaining = items.length;
```

```
    if (remaining === 0) return resolve(results);
    items.forEach((item, i) => {
      MyPromise.resolve(item).then((value) => {
        results[i] = value;
        if (--remaining === 0) resolve(results);
      }, reject);
    });
  });
}
```

`reject` can be called many times (every failing item calls it); the one-shot guard makes all but the first a no-op, so no extra flag is needed.

---

8. Implement in the correct order

#	Step	Working result
1	Constants + class with `#state`, `#value`	Shape in place.
2	Constructor: run executor, `resolve` / `reject` that set state once, `try/catch`	`new MyPromise(r => r(1))` settles; throws reject.
3	`then` that stores callbacks while pending (single level, sync)	Callbacks fire on settle.
4	Handler **queue** + flush in `#settle`	Many `then` calls on one promise work.
5	`then` returns a **child** promise; return value / throw feed the child	Chaining works.
6	Pass-through when a callback is missing	`catch` at the end of a chain works.
7	Wrap callbacks in `queueMicrotask`	Always async; ordering matches native.
8	Resolution procedure: adopt thenables, cycle check, read `then` once	Returning a promise from `then` waits for it.
9	One-shot guard via `#createResolvers`	Bad thenables cannot settle twice.
10	`catch`, `finally`, `resolve`, `reject`	Full instance API.
11	`all`, `allSettled`, `race`, `any`	Full static API.
12	Tests: self-check asserts + `promises-aplus-tests`	872/872 A+ tests passing.

---

9. Cover all states and edge cases

State / edge case	Behavior
Initial	`pending`, `#value` undefined, empty handler queue.
Processing	Executor running sync; async work keeps the promise pending.

State / edge case	Behavior
Fulfilled	`#value` holds the value; new `then` calls run their `onFulfilled` in a microtask.
Rejected	`#value` holds the reason; flows to the nearest `onRejected` / `catch`.
Adopting	`resolve(thenable)` called: still pending until the thenable settles; further `resolve` / `reject` ignored.
Executor throws after `resolve`	Ignored (already settled).
Executor not a function	Synchronous `TypeError`.
`resolve` called twice / `resolve` then `reject`	Only the first counts.
Callback returns the child promise	`TypeError` (chaining cycle).
`then` getter throws	Rejects with that error.
Thenable calls `resolve` then throws	Throw ignored.
Non-function arguments to `then`	Ignored; value / reason passes through.
`finally` callback returns a rejected promise	Result rejects with that reason.
`finally` with a non-function	Behaves like `then(fn, fn)`: pass-through.
`all([])` / `allSettled([])`	Resolve with `[]`.
`race([])`	Stays pending forever (same as native).
`any([])`	Rejects with `AggregateError`.
Non-iterable to statics	Returned promise rejects with `TypeError`.
Plain values in statics	Treated as already-fulfilled promises.
Mixed native + `MyPromise` inputs	Native promises adopted through their `then`.
Rejection with no handler	Silently lost (no unhandled rejection tracking: trade-off).

---

10. Final structure

```
Create a custom implementation of the Promise API/
├── index.js            # MyPromise class (+ module.exports)
├── check.js            # assert-based self-checks, incl. ordering vs native Promise
└── adapter.js          # adapter for the official promises-aplus-tests suite
```

Inside `MyPromise`:

```
MyPromise
├── #state, #value, #handlers
```

```
├── constructor
├── #createResolvers, #resolve, #settle, #runHandler   (private core)
├── then, catch, finally                               (instance API)
└── static resolve, reject, all, allSettled, race, any  (static API)
```

---

11. Full working code

`index.js`

```
const PENDING = 'pending';
const FULFILLED = 'fulfilled';
const REJECTED = 'rejected';

class MyPromise {
  #state = PENDING;
  #value = undefined;
  #handlers = []; // queued while pending: { onFulfilled, onRejected, resolve, reject }

  constructor(executor) {
    if (typeof executor !== 'function') {
      throw new TypeError(`Promise resolver ${executor} is not a function`);
    }
    const { resolve, reject } = this.#createResolvers();
    try {
      executor(resolve, reject);
    } catch (err) {
      reject(err); // a throw inside the executor rejects the promise
    }
  }

  // One-shot resolve/reject pair: only the first call of either one counts.
  #createResolvers() {
    let called = false;
    return {
      resolve: (value) => {
        if (called) return;
        called = true;
        this.#resolve(value);
      },
      reject: (reason) => {
        if (called) return;
        called = true;
        this.#settle(REJECTED, reason);
      },
    };
  }

  // Promise Resolution Procedure: adopt the state of thenables, fulfill with anything else.
```

```
  #resolve(x) {
    if (x === this) {
      return this.#settle(REJECTED, new TypeError('Chaining cycle detected for promise'));
    }
    if (x !== null && (typeof x === 'object' || typeof x === 'function')) {
      let then;
      try {
        then = x.then; // read once: a getter may throw or change between reads
      } catch (err) {
        return this.#settle(REJECTED, err);
      }
      if (typeof then === 'function') {
        const { resolve, reject } = this.#createResolvers();
        queueMicrotask(() => {
          try {
            then.call(x, resolve, reject);
          } catch (err) {
            reject(err); // ignored if the thenable already called resolve/reject
          }
        });
        return;
      }
    }
    this.#settle(FULFILLED, x);
  }

  #settle(state, value) {
    if (this.#state !== PENDING) return; // settled promises are immutable
    this.#state = state;
    this.#value = value;
    this.#handlers.forEach((handler) => this.#runHandler(handler));
    this.#handlers = [];
  }

  // Callbacks always run async (microtask), even if the promise is already settled.
  #runHandler({ onFulfilled, onRejected, resolve, reject }) {
    queueMicrotask(() => {
      const fulfilled = this.#state === FULFILLED;
      const callback = fulfilled ? onFulfilled : onRejected;
      if (typeof callback !== 'function') {
        // No handler for this state: pass the value / reason down the chain.
        return fulfilled ? resolve(this.#value) : reject(this.#value);
      }
      try {
        resolve(callback(this.#value));
      } catch (err) {
        reject(err);
      }
    });
  }
```

```
  then(onFulfilled, onRejected) {
    return new MyPromise((resolve, reject) => {
      const handler = { onFulfilled, onRejected, resolve, reject };
      if (this.#state === PENDING) this.#handlers.push(handler);
      else this.#runHandler(handler);
    });
  }

  catch(onRejected) {
    return this.then(undefined, onRejected);
  }

  finally(onFinally) {
    if (typeof onFinally !== 'function') return this.then(onFinally, onFinally);
    return this.then(
      (value) => MyPromise.resolve(onFinally()).then(() => value),
      (reason) =>
        MyPromise.resolve(onFinally()).then(() => {
          throw reason;
        }),
    );
  }

  static resolve(value) {
    if (value instanceof MyPromise) return value;
    return new MyPromise((resolve) => resolve(value));
  }

  static reject(reason) {
    return new MyPromise((_, reject) => reject(reason));
  }

  static all(iterable) {
    return new MyPromise((resolve, reject) => {
      const items = [...iterable]; // throws for non-iterables -> rejects
      const results = new Array(items.length);
      let remaining = items.length;
      if (remaining === 0) return resolve(results);
      items.forEach((item, i) => {
        MyPromise.resolve(item).then((value) => {
          results[i] = value; // keep input order, not completion order
          if (--remaining === 0) resolve(results);
        }, reject);
      });
    });
  }

  static allSettled(iterable) {
    return MyPromise.all(
      [...iterable].map((item) =>
        MyPromise.resolve(item).then(
```

```
          (value) => ({ status: FULFILLED, value }),
          (reason) => ({ status: REJECTED, reason }),
        ),
      ),
    );
  }

  static race(iterable) {
    return new MyPromise((resolve, reject) => {
      for (const item of iterable) MyPromise.resolve(item).then(resolve, reject);
    });
  }

  static any(iterable) {
    return new MyPromise((resolve, reject) => {
      const items = [...iterable];
      const errors = new Array(items.length);
      let remaining = items.length;
      if (remaining === 0) return reject(new AggregateError(errors, 'All promises were rejected'));
      items.forEach((item, i) => {
        MyPromise.resolve(item).then(resolve, (reason) => {
          errors[i] = reason;
          if (--remaining === 0) reject(new AggregateError(errors, 'All promises were rejected'));
        });
      });
    });
  }
}

module.exports = MyPromise;
```

`check.js` (self-checks)

```
const assert = require('node:assert/strict');
const MyPromise = require('./MyPromise');

const delay = (ms, v, fail) =>
  new MyPromise((res, rej) => setTimeout(() => (fail ? rej(v) : res(v)), ms));

// Same microtask ordering as native Promise
async function order(P) {
  const log = [];
  log.push('sync-1');
  P.resolve().then(() => log.push('then-1')).then(() => log.push('then-2'));
  new P((r) => {
    log.push('executor');
    r();
  }).then(() => log.push('then-a'));
  log.push('sync-2');
  await new Promise((r) => setTimeout(r, 10));
```

```
  return log;
}

(async () => {
  assert.deepEqual(await order(MyPromise), await order(Promise));

  assert.equal(await new MyPromise((r) => r(1)).then((x) => x + 1), 2);
  assert.equal(await MyPromise.reject(new Error('x')).catch((e) => e.message), 'x');
  assert.equal(await new MyPromise(() => { throw new Error('boom'); }).catch((e) => e.message),
'boom');
  assert.equal(await new MyPromise((r, j) => { r(1); j(2); r(3); }), 1); // first call wins
  assert.equal(await MyPromise.resolve(1).then().then(null).then((x) => x), 1); // pass-through
  assert.equal(await MyPromise.resolve(MyPromise.resolve(delay(5, 7))), 7); // adopt nested
  assert.equal(await MyPromise.resolve(Promise.resolve(9)), 9); // interop with native
  assert.equal(await MyPromise.resolve({ then: (r) => r(3) }), 3); // thenable

  const p = MyPromise.resolve(1);
  const cyc = p.then(() => cyc);
  await assert.rejects(cyc, TypeError);

  // finally: passes value through, waits for returned promise, overrides on throw
  let ran = false;
  assert.equal(await MyPromise.resolve(5).finally(() => { ran = true; return 99; }), 5);
  assert.ok(ran);
  await assert.rejects(MyPromise.reject(1).finally(() => {}), (e) => e === 1);
  await assert.rejects(MyPromise.resolve(1).finally(() => { throw 2; }), (e) => e === 2);

  assert.deepEqual(await MyPromise.all([delay(20, 'a'), 'b', delay(5, 'c')]), ['a', 'b', 'c']);
  assert.deepEqual(await MyPromise.all([]), []);
  await assert.rejects(MyPromise.all([delay(5, 1), delay(1, 'bad', true)]), (e) => e === 'bad');
  await assert.rejects(MyPromise.all(42), TypeError);

  assert.deepEqual(await MyPromise.allSettled([1, MyPromise.reject('no')]), [
    { status: 'fulfilled', value: 1 },
    { status: 'rejected', reason: 'no' },
  ]);
  assert.equal(await MyPromise.race([delay(20, 'slow'), delay(5, 'fast')]), 'fast');
  assert.equal(await MyPromise.any([MyPromise.reject(1), delay(5, 'ok')]), 'ok');
  await assert.rejects(MyPromise.any([MyPromise.reject(1), MyPromise.reject(2)]), (e) =>
    e instanceof AggregateError && e.errors.join() === '1,2');
  await assert.rejects(MyPromise.any([]), AggregateError);
  assert.throws(() => new MyPromise(), TypeError);

  console.log('all self-checks passed');
})();
```

`adapter.js` (Promises/A+ suite)

```
const MyPromise = require('./MyPromise');
```

```
exports.resolved = (v) => MyPromise.resolve(v);
exports.rejected = (r) => MyPromise.reject(r);
exports.deferred = () => {
  let resolve, reject;
  const promise = new MyPromise((res, rej) => { resolve = res; reject = rej; });
  return { promise, resolve, reject };
};
```

Run:

```
node check.js
npx -y promises-aplus-tests@2.1.2 adapter.js   # 872 passing
```

---

12. Wrap-up

Edge cases handled

- Settle-once, executor throws, missing executor.
- Always-async callbacks with native microtask ordering.
- Chaining, pass-through, promise adoption, native/thenable interop.
- Chaining cycles, throwing `then` getters, misbehaving thenables.
- `finally` keeps the value, waits for its promise, overrides on throw.
- Input order in `all`, empty inputs, non-iterables, `AggregateError` in `any`.

Trade-offs to mention

- `queueMicrotask` **vs** `setTimeout`**:** microtasks match native ordering and are faster; `setTimeout` is easier to explain but runs after timers and rendering.
- **Private** `#fields`**:** real encapsulation, but `instanceof MyPromise` in `resolve` means a subclass-unaware fast path; native uses `constructor` / `Symbol.species` checks.
- **No unhandled rejection tracking:** native warns when a rejection has no handler; ours drops it silently. Adding it needs a "handled" flag plus a deferred check.
- **One** `#value` **field for both value and reason:** less state; the `#state` field disambiguates.
- **Spread** `[...iterable]` **copies the input:** O(n) extra memory, but it gives the correct `TypeError` for non-iterables and a known length.

What to add with more time

- Unhandled rejection detection (`process.on('unhandledRejection')`-like hook).
- `MyPromise.withResolvers()` and `MyPromise.try()`.
- `Symbol.species` / subclassing so `class Ajax extends MyPromise` returns `Ajax` from `then`.
- `Symbol.toStringTag` so `Object.prototype.toString` prints `[object Promise]`.
- Cancellation via `AbortSignal` in a wrapper (promises themselves are not cancellable).
- Unit tests in Jest/Vitest with fake timers, plus a benchmark against native on a 100k-long chain.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why must the executor run synchronously but callbacks asynchronously?

The executor starts the work (e.g. a request), so it must run right away; that is native behavior too. Callbacks must be async so code after `.then()` always runs first, whether the value was already available or not. A predictable order prevents bugs that only appear when data is cached.

Q2. What is the difference between Promises/A+ and the ECMAScript Promise?

A+ only specifies `then` and the resolution procedure. ECMAScript adds the constructor, `catch`, `finally`, statics, `Symbol.species`, and the exact job ordering (e.g. thenables are called in a separate microtask). Our code passes A+ and matches ECMAScript ordering for the common cases.

Q3. What if the interviewer says "no `queueMicrotask` allowed"?

Fall back in this order: `process.nextTick` (Node), `MutationObserver` on a dummy text node (browsers, still a microtask), then `setTimeout(fn, 0)` (macrotask, works everywhere but changes ordering). Wrap it in one `schedule(fn)` helper so the choice lives in one place.

State design

Q4. Why not store `value` and `reason` separately?

Only one is ever set, and `#state` tells which one it is. Two fields add a way to be inconsistent (both set) with no benefit.

Q5. Why use an array of handlers instead of a single callback?

`then` can be called many times on the same pending promise (`p.then(a); p.then(b)`), and each needs to run in registration order. A single slot would overwrite earlier handlers.

Q6. Why `#private` fields instead of `this.state`?

With public fields any code could do `p.state = 'fulfilled'` and break the "settles once" guarantee. Private fields make the state machine enforce itself. Trade-off: debugging shows fewer internals, and pre-2022 runtimes need a `WeakMap` instead.

Q7. Why clear `#handlers` after settling?

Settled promises handle new `then` calls directly, so the old callbacks are never needed again. Keeping them would hold closures (and whatever they capture) in memory for as long as the promise lives.

Challenge: async callbacks

Q8. What is the output order?

```
setTimeout(() => console.log('timeout'));
MyPromise.resolve().then(() => console.log('micro'));
console.log('sync');
```

`sync`, `micro`, `timeout`. Sync code first, then the whole microtask queue, then the next macrotask (timer). Our self-check asserts that the microtask order matches native `Promise`.

Q9. Can microtasks starve the event loop?

Yes. If each microtask queues another (`function loop() { MyPromise.resolve().then(loop) }`), the queue never empties, so timers, I/O and rendering never run. Native promises have the same property; the fix is to yield with `setTimeout` in long loops.

Challenge: chaining

Q10. Why does `then` return a new promise instead of `this`?

Each step in a chain has its own result. If `then` returned `this`, `p.then(x => x + 1).then(y => ...)` would give `y` the original `x`, and errors could not be caught per step. Separate promises also let one promise branch into several independent chains.

Q11. How does a rejection skip `then(onOk)` and reach `catch`?

When the matching callback is missing, `#runHandler` forwards the reason with `reject(this.#value)`. Each `then` without `onRejected` passes it on until a promise with an `onRejected` (a `catch`) handles it.

Q12. What happens if `catch` returns a value?

The chain **recovers**: the promise returned by `catch` fulfills with that value, so later `then` calls run normally. To keep it failing, `catch` must rethrow.

Challenge: resolution procedure

Q13. Why read `x.then` only once?

It can be a getter with side effects or one that returns different functions each time; reading it twice could check one function and call another. The spec requires reading it once and calling `then.call(x, ...)`.

Q14. Why a fresh one-shot `resolve` / `reject` pair for each thenable?

A third-party thenable might call `resolve` twice, call both, or call one and then throw. The fresh pair's `called` flag makes only the first call count, so a buggy library cannot settle our promise twice or turn a fulfillment into a rejection.

Q15. What if `resolve` receives a promise that rejects?

The outer promise **rejects** with that reason: `resolve(p)` means "become whatever `p` becomes", not "fulfill with `p`". That is why `resolve` is better named "adopt". `reject(p)` never unwraps: the reason is the promise object itself.

Q16. Why not detect longer cycles (A waits for B, B waits for A)?

The spec only requires the direct self-reference check. Longer cycles would need graph tracking on every adoption, costing memory and time on the hot path for a rare bug; native promises also just hang in that case.

Challenge: combinators

Q17. Why not use `results.push` in `all`?

`push` stores results in **completion** order. `all` promises input order, so we write `results[i]` with the index captured in a closure.

Q18. Why count `remaining` instead of checking `results.length === items.length`?

`new Array(n)` already has length `n`, and writing `results[2]` first also makes the length 3. A countdown is the only reliable "all done" signal.

Q19. What if one input to `all` never settles?

`all` never settles either: there is no built-in timeout. Wrap it: `MyPromise.race([MyPromise.all(list), timeout(5000)])`, where `timeout` rejects after the delay.

Q20. What if `all` gets 1 million promises?

It works in O(n) time and memory: one handler and one array slot per item. The real problem is usually starting 1M requests at once; use a concurrency-limited pool (e.g. 10 at a time) that starts the next task when one finishes.

Q21. How is `any` different from `race`?

`race` settles with the first input to settle, fulfilled **or** rejected. `any` ignores rejections until every input has rejected, then rejects with an `AggregateError` listing all reasons. Use `any` for "first mirror that responds successfully".

Implementation steps

Q22. Why build sync callbacks first and add `queueMicrotask` later?

It gets chaining working with an easy-to-debug call stack, then one line changes the timing. Each layer is testable on its own, which matters if time runs out.

Q23. How do you test it?

Two layers: assert-based checks for our API and the ordering vs native `Promise`, and the official `promises-aplus-tests` suite through a small `deferred()` adapter. 872 tests pass. In a real project, Jest/Vitest with fake timers for the timer-based cases.

Q24. How does `await` work with `MyPromise`?

`await x` calls `Promise.resolve(x)` under the hood, which adopts any thenable by calling `x.then(resolve, reject)`. Because our `then` follows the spec, `await` works without any special code.

Performance

Q25. What is the cost of a long chain of 100k `then` calls?

O(n) memory (one promise and one closure per link) and one microtask per link, so it finishes in milliseconds. No recursion on the call stack: each link runs in its own microtask, so there is no stack overflow.

Q26. Why `forEach` + `queueMicrotask` per handler instead of one microtask for all?

Simpler and matches native (one job per reaction). Batching into one microtask would save a few allocations but changes the interleaving with other promises' callbacks.

Trade-offs: "why not X?"

Q27. Why not extend the native `Promise` (`class MyPromise extends Promise`)?

That skips the whole point: the interview tests whether you understand states, the handler queue and the resolution procedure. Extending native is the right choice in production when you only need extra methods.

Q28. Why not `setTimeout` for scheduling?

It runs callbacks after timers, I/O and rendering, so ordering differs from native and every link adds a ~1-4 ms delay in browsers. `queueMicrotask` is the same queue native promises use.

Q29. Why not `Array.from(iterable)` in the statics?

`Array.from(42)` returns `[]` (it accepts array-likes), so `all(42)` would resolve instead of rejecting. Spreading `[...iterable]` throws `TypeError` like native, and the executor's `try/catch` turns that into a rejection.

Q30. How would you make a promise cancellable?

Promises have no cancel by design (many consumers share one result). Pass an `AbortSignal` to the work that creates it: the executor listens for `abort`, stops the work, and rejects with an `AbortError`.

```
const wait = (ms, signal) => new MyPromise((res, rej) => {
  const id = setTimeout(res, ms);
  signal.addEventListener('abort', () => { clearTimeout(id); rej(signal.reason); });
});
```